Capital Circuit - Monopoly Game

CSC301 Group 4 | console Monopoly in Java (Abu Dhabi edition)

Data structures used in the Monopoly Game and its explanation

Course	CSC301 Data Structures and Algorithms - Dr Zakaria Khiati, Abu Dhabi University
Group / member	Group 4 | CS (Caron Sylvia)
Code base	27 Java files in C:\Claude\Monopoly\code (default package), board and card files in code\data; branch CS-Monopoly of CSC301-Group4-Monopoly
Scope	Core game (brief Section 3) with console board view, colours and standings, plus the group's three twists: T1 Reverse, T3 Expansion, T7 Recall (Appendix A, Section 3 of this report). Bonus B4 and language toggle planned
Verification	93 automated tests (incl. every Section 8.2 scenario and the T1 scenario), 1,000 seeded games, automatic constraint check incl. the twist methods: PASS
Brief reference	Section 4 (required data structures), 4.1, 4.2, 5 and 5.1 (classes, recursive methods), 6 and Appendix A (twists), 8.1 (what every member must understand), 9 (design document)

1. Summary

1.1 Overview

The game is a console version of simplified Monopoly on the group's own 24-square Abu Dhabi board. It loads the board and cards from text files, lets 2 to 6 players take turns with two dice, and applies every rule in Section 3 of the brief: buying, rent (doubled for a full colour group), transit rent, tax, Chance and Community cards, Jail, doubles, bankruptcy, the end of the game and the final ranking. Each event is written to the Appendix C game log. A colour board view shows the squares, owners, tokens, each player's cash and properties, the turn queue and the last turn's events.

The brief fixes which data structure does which job, and the code follows it exactly. The **board** is the group's own **circular doubly linked list**: one node per square, the last node linked back to GO, and every player's position held as a node reference rather than an index. **Turn order** and the two **card decks** use the group's own **linked queue** on singly linked nodes. **ArrayList** holds data that needs indexed access and a changing size: each player's squares, the list of players for the ranking and the card list before shuffling. **Recursion**, with no loops, implements the eight required methods for movement, doubles, landing, rent, net worth and board searches.

The group's three **twists** each stress one of these structures. **T1 Reverse** reverses the turn queue with a recursive method that may use only enqueue, dequeue and isEmpty. **T3 Expansion** grows the ring from 24 to 27 squares at the end of turn 20 with insertAfter, without moving any player. **T7 Recall** rotates a card deck recursively so that its last three cards come to the front. Section 3 explains each one: rules, pointer diagrams, code, termination, edge cases and how it changes the standard flow.

2	4	24 > 27	8	2 + 12	0
own generic structures	LinkedQueue instances	board nodes (T3 at turn 20)	required recursive methods	twist methods + recursive helpers	banned collections used

1.2 Data structures at a glance

Data structure	Implementation in the code	Game functions it serves	Key operations and cost
Circular doubly linked list	Own generic `CircularDoublyLinkedList<T>` with `Node<T>` (data, next, prev); wrapped by `Board`	The board; player positions; dice and card movement forward and backward; salary at GO; colour-group check; MOVE_TO / NEAREST searches; **T3: three squares inserted after turn 20**; drawing the board	addLast, insertAfter, remove, getHead, size: O(1). Moving k squares: O(k). Searches: O(n)
Queue (singly linked)	Own generic `LinkedQueue<T>` with private `QNode<T>` (data, next): `Game.turnQueue`	Turn order; removing a bankrupt player; the order of PAY_EACH and COLLECT_EACH; the game-over check; **T1: reversed by a REVERSE card**	enqueue, dequeue, peek, isEmpty, size: O(1). remove(p), copyInto, reverse: one pass, O(n)
Queue (same class)	`Game.chance`, `Game.community`	Chance and Community decks: draw from the front, return to the back; JAIL_FREE cards returned to their deck; **T7: last 3 cards recalled to the front**	Draw and return: O(1). Recall: rotate(n - 3), O(n)
Queue (same class)	`Board.pendingExpansions`: `LinkedQueue<Expansion>`	**T3:** EXPAND lines held in file order until the end of turn 20, then dequeued one by one and inserted	O(1) per line
ArrayList	`java.util.ArrayList`	Each player's owned squares and held JAIL_FREE cards; all players (join order, ranking); card list before shuffling; bankruptcy order; log lines; scripted dice	get: O(1); add and remove-last: O(1) amortised; sort: O(n log n)
Recursion	8 required methods + 2 twist methods + 12 helpers, no for/while/do	Movement, doubles, landing and card chains, rent, net worth, board searches, queue rotation, bankruptcy clean-up; **reverse (T1), rotate (T7), applyExpansions (T3)**	See Sections 2.4 and 3.6

1.3 The game on screen

Screenshot taken by CS from the running game in the Windows Command Prompt: turn 18 of a three-player game (Caron, Helen, Jay), just after Q was pressed (before the twists were added; the board is the 24-square starting board). The centre shows the standings; the report under the board lists each player's net worth and properties.



What you see on screen	Data structure behind it
The ring of squares, drawn clockwise from GO in the top-left corner (7 x 7 for 24 squares, 8 x 8 after the T3 expansion)	**Circular doubly linked list**: BoardView walks next from the head (GO) once round and places each square on the screen

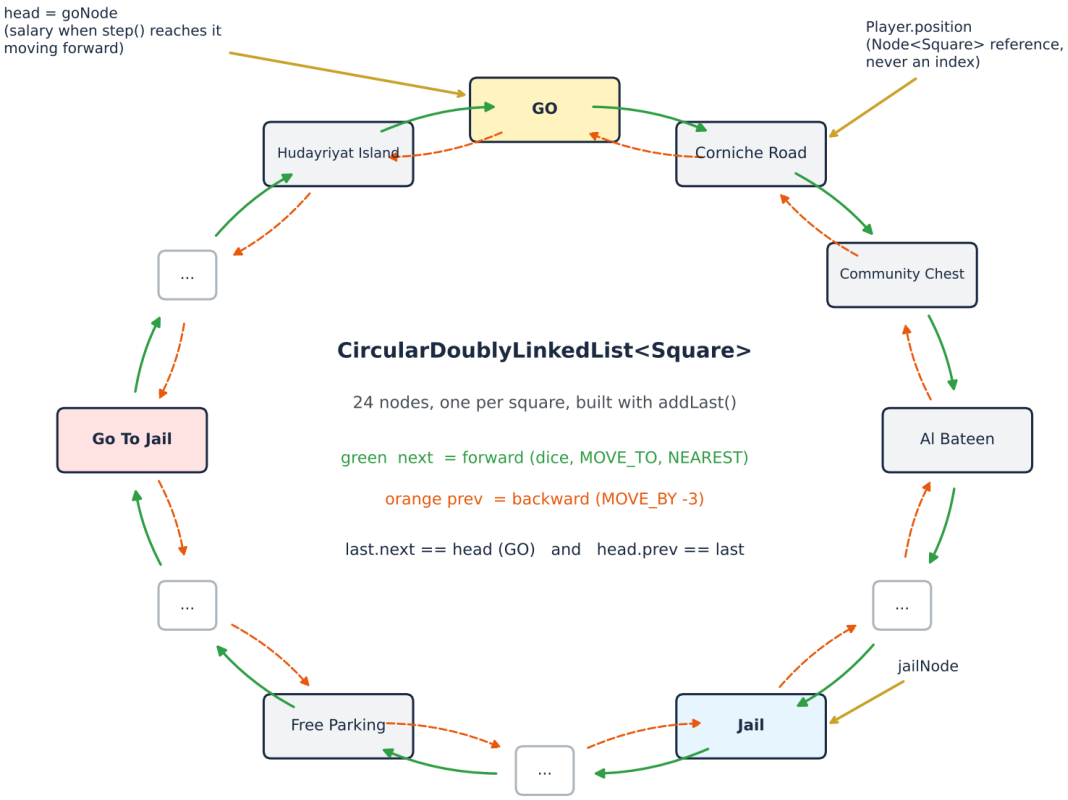
What you see on screen	Data structure behind it
A player's token (C, H, J) on a square	`Player.position`, a `Node<Square>` reference into the ring
Owner marker [C] / [H] / [J], square tinted in the owner's colour	`OwnableSquare.owner` on the node's square, plus the owner's **ArrayList** of squares
"owns: ..." lines and the property list in the report	Board walked in ring order and filtered by owner; count from `Player.owned` (**ArrayList**)
Ranking by net worth, with cash and property value	**ArrayList** of players sorted with `Collections.sort` (Player implements Comparable); property value by recursive `netWorth(owned, 0)`
Turn queue line and "Next to play" (during play; reversed after a REVERSE card)	**LinkedQueue<Player>**: `peek()` and a one-rotation `copyInto()`
Events of the last turn, incl. twist events ("New turn order: D > C > B")	Appendix C log lines kept in an **ArrayList<String>** by GameLogger and turned into sentences

2. Detailed explanation

2.1 Circular doubly linked list - the board

What it is. Each `Node<Square>` holds one square and two links: next (the following square) and prev (the one before). The last square's next is GO and GO's prev is the last square, so the ring has no ends: a player keeps going round without any index arithmetic or % size, which the brief forbids for movement (4.2).

Why this structure. Every move in Monopoly is "go to the next square" or "go back one square", which is a single pointer hop in either direction. Passing GO is detected by node identity (the next node is goNode). Positions are node references, so when squares are inserted during play (our twist T3 Expansion) no player's position has to be renumbered. An ArrayList was rejected because movement would need index arithmetic and inserting a square would shift every later index (O(n)); a singly linked list was rejected because backward moves (MOVE_BY -3) would need a full lap, and inserting after a node would still need its successor's prev link for backward moves.



The starting ring of 24 squares. After turn 20 the T3 expansion adds three nodes (Section 3.3).

Operations (brief 4.1)

Operation	What it does	Used for in the game	Cost
addLast(T)	Links a new node between the last node and GO	FileLoader builds the board in file order, GO first	O(1)
insertAfter(Node, T)	Links a new node directly after a given node	**T3 Expansion**: Al Mina, Abu Dhabi Airport Express and Lulu Island inserted at the end of turn 20	O(1)
remove(Node)	Unlinks a node; if it is the head, the head moves on	Required by brief 4.1 and tested; not used in play (T2 Earthquake is not one of our twists)	O(1)
getHead()	Returns the GO node	Start of every search and of the board drawing	O(1)
size()	Number of squares (live: 24, then 27)	Bound for searches (remaining), screen layout	O(1)

Game functions built on the ring

- **Board set-up** - `FileLoader.loadBoard` reads each line, creates the right Square subclass and calls addLast; Board keeps goNode (the head) and finds jailNode with the recursive findNearest. EXPAND lines are parsed by the same code but held back in a queue (T3).
- **Dice movement and salary** - `Game.movePlayer` calls the recursive `Board.step(current, total, p)`, which follows next one square per call and pays 200 when the next node is GO (onto or past GO). Then the MOVE line is logged, followed by SALARY.
- **Backward moves** - a negative step count (card "Go back 3 squares") follows prev and never pays a salary.
- **MOVE_TO and NEAREST cards** - findByName or findNearest locates the target node; the recursive forwardDistance counts the steps; step then moves forward, so passing GO pays the salary (card K9 goes a full lap).
- **Go to Jail** - the player's position is set straight to jailNode: no walk along the ring, so no salary.
- **Rent for a full colour group** - the recursive ownsGroup walks once round from GO and stops at the first square of that colour the owner does not hold. It walks the live ring, so a square added by T3 counts at once.
- **T3 Expansion** - at the end of turn 20, insertAfter links each new square after its named square (Section 3.3).
- **Board view** - BoardView walks next from GO to place the squares on screen; display only, never used for movement.

Code excerpt - Board.step (required recursive method)

code\Board.java

```
/**
 * Moves steps squares: along next if positive, along prev if negative.
 * Pays the salary (and flags it on p) when moving forward onto or past GO.
 * Base case: steps == 0 returns current.
 * Recursive case: one step towards zero, then recurse with steps - 1 or steps + 1.
 * Terminates because |steps| falls by 1 each call.
 */
public Node<Square> step(Node<Square> current, int steps, Player p) {
    if (steps == 0) {
        return current;
    }
    if (steps > 0) {
        Node<Square> next = current.next;
        if (next == goNode && p != null) {
            p.addCash(SALARY);
            p.setSalaryPaid(true);
        }
        return step(next, steps - 1, p);
    }
    return step(current.prev, steps + 1, p);
}
```

2.2 Linked queue - turn order, card decks and pending expansions

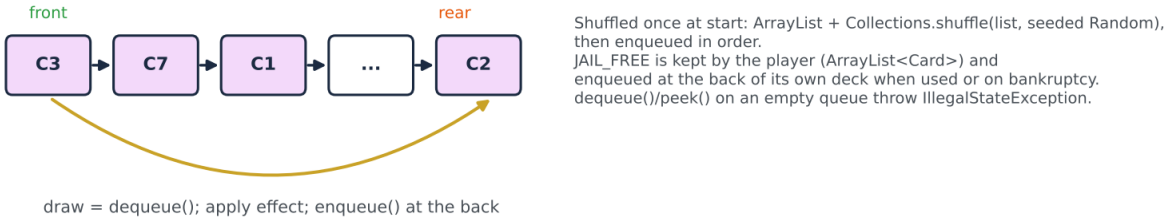
What it is. A first-in, first-out queue on singly linked nodes, with front and rear references. The same generic class is used four times: `LinkedQueue<Player>` for the turn order, two `LinkedQueue<Card>` for the decks and `LinkedQueue<Expansion>` for the EXPAND lines waiting for turn 20 (T3).

Why this structure. All these jobs are naturally FIFO: the player at the front plays and goes to the back; the top card is drawn and goes to the bottom; EXPAND lines are applied in file order. Both ends are O(1). An ArrayList was rejected because removing from the front shifts every element (O(n)); Java's ArrayDeque and LinkedList are banned by the brief.

Turn order - LinkedQueue<Player>



Card decks - LinkedQueue<Card> (Chance and Community)



Game functions built on the queue

- **A turn** - `Game.playOneTurn`: dequeue() the front player, log TURN, apply the Jail rules or takeTurn, then enqueue() the player at the back unless bankrupt. At the end of turn 20 the pending expansions are applied (T3). The game ends when turnQueue.size() <= 1 (LAST_PLAYER) or the turn limit is reached.
- **Removing a bankrupt player** - remove(p) rotates the queue exactly once with dequeue and enqueue (recursively) and drops p on the way, so the other players keep their order (brief 3.7 and 8.2).
- **PAY_EACH and COLLECT_EACH** - copyInto rotates the queue once to read the other players in turn-queue order (the current player is not in the queue during the turn); the payments then recurse over that list. After a REVERSE card they follow the reversed order.
- **Decks** - cards are loaded into an ArrayList, shuffled once with the seeded Random (Chance first, then Community) and enqueued. Drawing is dequeue(), apply, enqueue() at the back. A JAIL_FREE card stays in the player's jailFreeCards list and returns to the back of its own deck when used or on bankruptcy.
- **T1 Reverse** - `QueueTwists.reverse(turnQueue)`: dequeue one player, reverse the rest recursively, enqueue that player (Section 3.2).
- **T7 Recall** - `QueueTwists.rotate(deck, n - 3)` moves cards from the front to the back one at a time (Section 3.4).
- **Empty queue** - dequeue() and peek() throw IllegalStateException with a clear message (brief 4.1).

Code excerpt - LinkedQueue.remove (order-preserving removal)

code\LinkedQueue.java

```
/**
 * Removes the first occurrence of item (by reference) and keeps the order of
 * all other elements. Rotates the queue exactly once using only dequeue and
 * enqueue. Recursive: base case remaining == 0. O(n).
 */
public boolean remove(T item) {
    return removeRotate(item, size, false);
}

private boolean removeRotate(T item, int remaining, boolean removed) {
    if (remaining == 0) {
        return removed;
    }
    T x = dequeue();
    if (!removed && x == item) {
        removed = true;
    } else {
        enqueue(x);
    }
    return removeRotate(item, remaining - 1, removed);
}
```

2.3 ArrayList - indexed data with a changing size

The brief reserves java.util.ArrayList for data that needs indexed access and a changing size. Recursion over an ArrayList uses an index parameter, which is why `netWorth(owned, index)` takes one.

ArrayList	Where	Game function	Why ArrayList
ArrayList<Square> owned	Player	Buying adds a square; transit rent counts the owner's TRANSIT squares (recursive countOwned); net worth sums prices (recursive netWorth); bankruptcy returns every square (recursive, removing the last element each call, O(1))	Indexed recursion; grows as squares are bought
ArrayList<Player> players	Game	Join order; final ranking with Collections.sort (Player implements Comparable: higher net worth first, ties by join order - not by queue order, even after T1); board view	Sorting and index access; java.util.Comparator is not on the allowed list
ArrayList<Card> (card list before shuffling)	FileLoader, Game	Cards in file order, split per deck, Collections.shuffle(list, seeded Random), then enqueued	Collections works on ArrayLists (brief 4.2)
ArrayList<Card> jailFreeCards	Player	Held Get Out of Jail Free cards; used first-in at the next jailed turn; not counted by RECALL (T7)	Small list that changes size
ArrayList<Player> bankruptOrder	Game	Ranking lists bankrupt players most recent first	Read backwards by index
ArrayList<Expansion> (load time only)	FileLoader	T3: EXPAND lines collected while the file is read, then enqueued in file order on the Board	Count check (24 + 3 <= 28) before the Board exists
ArrayList<String> lines	GameLogger	Every Appendix C line, for the tests and the board view's event list	Append and read by index
ArrayList<Integer> scripted	Dice	Scripted dice values (--dice) for tests and demo scenarios	Read in order by index

2.4 Recursion - the eight required methods (brief 5.1)

None of these methods contains a for, while or do loop, and every helper they call is recursive or O(1). This is checked automatically by tests\check_constraints.py. The two twist methods (reverse, rotate) are in Section 3.6.

Method (class)	Game function	Base case	Recursive case	Why it terminates
step (Board)	Move by dice or card; pay salary at GO	steps == 0: return current	One hop along next (or prev), then step(.., steps - 1) (or + 1)	|steps| falls by 1 each call
takeTurn (Game)	One roll, again after a double	Third double: Jail, no move. Jailed or bankrupt after landing. Not a double	A double: takeTurn(p, doublesCount + 1)	doublesCount rises each call; the third double always stops
landOn (Game)	Resolve the square; follow card moves	Square did not move the player by a card, or player jailed/bankrupt	A card moved the player: landOn(p)	No card move ends on a card square, before or after T3 (tested): depth at most 2
transitRent (TransitSquare)	Rent 25 x 2^(n-1)	owned <= 1: 25	2 x transitRent(owned - 1)	owned falls by 1 to 1

Method (class)	Game function	Base case	Recursive case	Why it terminates
netWorth (Player)	Property value for ranking	index >= size: 0	price at index + netWorth(owned, index + 1)	index rises to the list size
ownsGroup (Board)	Double rent for a full colour group	A property of the colour not owned by p: false. Back at the start: true	Check current, recurse on current.next	The ring is finite; the walk stops at start
findByName (Board)	MOVE_TO cards; card-file validation; T3 anchor square	remaining == 0: null. Name matches: node	Recurse on next, remaining - 1	remaining falls by 1
findNearest (Board)	NEAREST cards; finding Jail at set-up	remaining == 0: null. Type matches: node	Recurse on next, remaining - 1	remaining falls by 1

Recursive helpers

Helper	Purpose
LinkedQueue.removeRotate, copyRotate	Remove a player / read the queue in order with one rotation, order unchanged
Board.countForward (via forwardDistance)	Steps forward to a MOVE_TO or NEAREST target
Player.countOwned	Number of TRANSIT squares owned, for transit rent
Game.payEach, collectEach	PAY_EACH / COLLECT_EACH in turn-queue order; stop or bankrupt when someone cannot pay
Game.releaseSquares, returnJailFreeCards, clearCards	Bankruptcy: squares back to the bank, JAIL_FREE cards back to their decks
ConsoleBuyDecider.ask	Re-ask Y/N on invalid input without a loop inside landOn
Game.applyExpansions (T3)	Dequeue each pending EXPAND line and insert it; base case: queue empty
Game.fillNames (T1)	Build the ORDER log line from the reversed queue, one name per call

Code excerpt - Game.landOn (required recursive method)

code\Game.java

```
/**
 * Required recursive method. Resolves the square the player stands on and
 * calls itself when a card moved the player to another square.
 * Base case: the square did not move the player by a card, or the player
 * is now in Jail or bankrupt.
 * Recursive case: a card moved the player -> landOn(p) on the new square.
 * Terminates because each recursive call needs a card move, and the card
 * file is checked so that no card move ends on a CHANCE or COMMUNITY
 * square: depth is at most 2.
 */
public void landOn(Player p) {
    p.setMovedByCard(false);
    p.getPosition().data.land(this, p);
    if (p.isMovedByCard() && !p.isBankrupt() && !p.isInJail()) {
        landOn(p);
    }
}
```

2.5 Supporting Java constructs

Construct	Where	Role
Abstract class and inheritance	Square > OwnableSquare > PropertySquare, TransitSquare; TaxSquare, CardSquare, CornerSquare	Each square type decides what happens on landing through land(Game, Player); landOn calls it polymorphically
Enumerations	DeckType, CornerType, CardEffect (incl. REVERSE, RECALL)	Fixed sets of values; CardEffect drives a switch in Game.applyCard
Static generic methods	QueueTwists.reverse, QueueTwists.rotate	Signatures fixed by Appendix A (static <T> void ...); use only the queue's public operations
Interface	BuyDecider: ConsoleBuyDecider (Y/N), AutoBuyDecider (--auto)	"The player decides whether to buy" (brief 3.4), interactive or scripted
java.util.Random (seeded)	Dice; Collections.shuffle	Same seed gives the same dice and card order, so the same game (brief 3)
java.util.Scanner	Main, ConsoleBuyDecider	Keyboard input

Construct	Where	Role
java.io	FileLoader (BufferedReader), GameLogger (PrintWriter)	Board and card files in; game log out
Plain Java arrays	BoardView screen buffer (char[][], String[][]); log fields (String[])	Drawing the screen and building one log line only; never holds game state

2.6 Compliance with the constraints (brief 4.2 and Appendix A)

Rule	How the code meets it	Evidence
Own CircularDoublyLinkedList<T> and LinkedQueue<T>	Both written by the group with Node / QNode; no Java collection inside	Source; UML class diagram (Section 4)
A position is a Node<Square>; no index arithmetic or % size for movement	Player.position is a node; only Game.movePlayer and Game.sendToJail change it; T3 never touches positions	Constraint check: no "% size"; T3 test: same nodes before and after
From java.util only ArrayList, Scanner, Random, Collections	Imports limited to these four (plus java.io)	Constraint check: imports
No ArrayDeque, Stack, Vector, PriorityQueue, Deque, HashMap, TreeMap, HashSet, other collections, streams	None used; Comparable (java.lang) replaces java.util.Comparator	Constraint check: banned names
Required methods recursive, no loops; helpers recursive or O(1)	8 methods + 12 recursive helpers	Constraint check; design note D18
T1: reverse recursive, only enqueue, dequeue, isEmpty; no other list, array or stack	QueueTwists.reverse: three calls, nothing else	Constraint check (queue operations used; no new / [] in the body)
T7: rotate recursive, moves k cards front to back	QueueTwists.rotate: enqueue(dequeue()), then rotate(q, k - 1)	Constraint check; T7 tests
dequeue()/peek() on an empty queue throw a clear exception	IllegalStateException("... cannot dequeue from an empty queue")	Tests 1-2

2.7 Data flow through one turn (brief 8.1)

Step	Class.method	What happens	Data structure
Set-up	Main, FileLoader	Board file to Square objects; EXPAND lines held back (T3); cards to a list; players added in join order	CDLL (addLast); LinkedQueue<Expansion>; ArrayList; shuffle; LinkedQueue (enqueue)
1	Game.playOneTurn	Front player taken; TURN logged	LinkedQueue.dequeue
2	Game.jailTurn (if jailed)	Use JAIL_FREE, or roll: double releases, third failure pays 50	ArrayList (held card); deck LinkedQueue.enqueue
3	Game.takeTurn	Roll two dice; ROLL logged; third double sends to Jail with no move	Recursion (doublesCount)
4	Game.movePlayer, Board.step	Walk the ring; salary at GO; MOVE then SALARY logged	CDLL next / prev; recursion
5	Game.landOn, Square.land	Buy, rent, tax, card or Go to Jail, by square type	Polymorphism; recursion for card moves
5a	landOnOwnable	Buy (owned.add) or pay rent (ownsGroup / transitRent)	ArrayList; CDLL walk; recursion
5b	drawCard, applyCard	Card drawn and returned; may move, pay, collect or jail; REVERSE (T1) reverses the waiting players; RECALL (T7) rotates its deck	Deck LinkedQueue; turn LinkedQueue; recursion
6	declareBankrupt (if needed)	Cash to creditor; squares to bank; cards to decks; removed from queue	ArrayList (recursive); LinkedQueue.remove
7	Game.takeTurn	Double: roll again	Recursion
8	Game.playOneTurn	Player enqueued at the back unless bankrupt; at the end of turn 20 EXPAND lines inserted (T3); end-of-game check	LinkedQueue.enqueue, size; CDLL insertAfter
End	Game.finish, rank	END and RANK lines; standings with properties	ArrayList sort; recursion (netWorth); CDLL walk

2.8 Game functionality and the data structure behind it

A one-line answer for every function built, as asked in brief Section 8.1 ("the role of each data structure ... and the cost of its operations").

Game functionality built	Data structure	Algorithm / method	Cost
Load the 24-square board	Circular doubly linked list	addLast per line; checks for one GO, JAIL, GO_TO_JAIL	O(n)
Load and shuffle the cards	ArrayList then LinkedQueue	Collections.shuffle (seeded), then enqueue	O(n)
Add players; turn order	ArrayList + LinkedQueue	add; enqueue in join order	O(1) each
Start a turn	LinkedQueue	dequeue the front player	O(1)
Move by the dice	Circular doubly linked list	recursive step along next	O(k)
Salary for passing or landing on GO	Circular doubly linked list	next == goNode inside step	O(1) per step
Move backward (card)	Circular doubly linked list	step with prev, no salary	O(k)
MOVE_TO / NEAREST cards	Circular doubly linked list	recursive findByName / findNearest + forwardDistance	O(n)
Go to Jail (square, card, third double)	Circular doubly linked list	direct jump to jailNode	O(1)
Doubles; third double	Recursion	takeTurn(p, doublesCount + 1)	at most 3 calls
Resolve a landing; card chains	Recursion + polymorphism	landOn then Square.land	depth at most 2
Buy a square	ArrayList	owned.add; owner set on the square	O(1)
Rent with colour-group doubling	Circular doubly linked list	recursive ownsGroup once round	O(n)
Transit rent	ArrayList + recursion	countOwned then transitRent	O(m)
Draw and return a card	LinkedQueue	dequeue, apply, enqueue	O(1)
Get Out of Jail Free	ArrayList + LinkedQueue	kept in player list; enqueued to its deck when used	O(1)
Pay each / collect each	LinkedQueue + ArrayList + recursion	copyInto rotation; recursive payEach / collectEach	O(p)
Jail attempts and fine	Player fields	attempt counter; third failure pays 50	O(1)
Bankruptcy	ArrayList + LinkedQueue	recursive release of squares and cards; remove by rotation	O(n)
T1 Reverse turn order	LinkedQueue + recursion	QueueTwists.reverse; ORDER via copyInto	O(p)
T3 Board expansion at turn 20	LinkedQueue + circular doubly linked list	dequeue each EXPAND line; findByName; insertAfter	O(e x n)
T7 Recall last 3 cards	LinkedQueue + recursion	QueueTwists.rotate(deck, n - 3)	O(c)
End of game	LinkedQueue	size() <= 1 or turn limit	O(1)
Net worth and final ranking	ArrayList + recursion	netWorth recursion; Collections.sort with Comparable	O(p log p)
Game log (Appendix C)	ArrayList + file	GameLogger lines; PrintWriter	O(1) per line
Colour board view	Circular doubly linked list (read only)	walk next from GO into a screen grid	O(n)
Standings with properties	Circular doubly linked list + ArrayList	walk the ring filtered by owner; rank()	O(n x p)
Same game for the same seed	Random (seeded); ArrayList of scripted dice	one Random for dice and shuffle	-

n = squares on the board (24, then 27), k = squares moved, p = players (2-6), m = squares a player owns, c = cards in a deck (12), e = EXPAND lines (3).

3. Twists - T1 Reverse, T3 Expansion, T7 Recall

Brief Section 6 asks each group to implement three twists from the pool in Appendix A. Group 4 implements **T1 Reverse** (queue, recursion), **T3 Expansion** (doubly linked list) and **T7 Recall** (queue, recursion). Together they exercise all three structures the brief grades: the turn queue, the card-deck queues and the board ring. Each twist was built on an extension point left in the core design (design note D11): two new CardEffect values with one case each in `Game.applyCard`, and the existing `insertAfter` of the list. The twist assignment is still to be confirmed against the group's Blackboard list.

3.1 The three twists at a glance

Twist	Structure stressed	Trigger	What changes	New code	Log lines (Appendix C format)
T1 Reverse	LinkedQueue<Player> turnQueue; recursion	Card effect REVERSE (cards C11 Chance, K11 Community)	The waiting players are reversed; the current player joins the back as usual, so play runs the other way	QueueTwists.reverse; Game.reverseTurnOrder, fillNames	REVERSE|<player> then ORDER|<p1>|<p2>|... (front first)
T3 Expansion	CircularDoublyLinkedList<Square>; LinkedQueue<Expansion>	End of turn 20 (EXPAND lines at the end of board.txt)	Three squares are linked into the ring: 24 to 27 squares. Nobody moves	Expansion; FileLoader EXPAND parsing; Board.insertAfter; Game.applyExpansions	EXPAND|<new square>|<after square>, one per line, in file order
T7 Recall	LinkedQueue<Card> deck; recursion	Card effect RECALL (cards C12 Chance, K12 Community)	The last 3 cards of the same deck move to the front, in order; RECALL then goes to the back	QueueTwists.rotate; Game.recall	RECALL|<deck>|<id1>|<id2>|<id3> (front first)

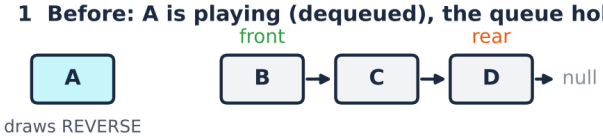
3.2 T1 Reverse - reversing the turn queue

Rule (Appendix A). A REVERSE card reverses the order of the turn queue, so play continues in the opposite direction. The current player is not in the queue during their turn and is enqueued at the back as usual when the turn ends. The reversal must be done by a recursive `static <T> void reverse(LinkedQueue<T> q)` that uses only enqueue, dequeue and isEmpty; no other list, array or stack may be used.

How it works. `reverse` dequeues the front player, reverses the rest recursively, and enqueues that player at the back. The players wait on the call stack instead of in a second structure, so the method meets the "no other list, array or stack" rule while still needing O(n) temporary storage. `Game.reverseTurnOrder` logs REVERSE, calls reverse, then reads the new order with one copyInto rotation to print the ORDER line.

T1 Reverse - LinkedQueue<Player> turnQueue (brief 8.2 scenario)

1 Before: A is playing (dequeued), the queue holds B, C, D



2 QueueTwists.reverse(q): each call dequeues one player; the call stack holds them



3 After: REVERSE|A, ORDER|D|C|B; A is enqueued at the back when the turn ends



Next four turns: D, C, B, A

Cost: O(n) queue operations

Code excerpt - QueueTwists.reverse and Game.reverseTurnOrder

code\QueueTwists.java, code\Game.java

```
/**
 * T1 Reverse: reverses the order of q.
 * Base case: q is empty - nothing to reverse.
 * Recursive case: take the front element x, reverse the rest, then put x
 * at the back. x was first, so it ends up last; by induction the rest is
 * already reversed in front of it.
 * Terminates because each call removes one element, so the queue reaches
 * empty after size() calls (depth = size, at most 5 players here).
 * Cost: O(n) operations, O(n) call-stack frames.
 */
public static <T> void reverse(LinkedQueue<T> q) {
    if (q.isEmpty()) {
        return;
    }
    T x = q.dequeue();
    reverse(q);
    q.enqueue(x);
}

private void reverseTurnOrder(Player p) {
    logger.log("REVERSE", p.getName());
    QueueTwists.reverse(turnQueue);
    ArrayList<Player> order = new ArrayList<>();
    turnQueue.copyInto(order);              // one rotation, order unchanged
    String[] fields = new String[order.size() + 1];
    fields[0] = "ORDER";
    fillNames(order, 0, fields);
    logger.log(fields);
}
```

Edge cases and knock-on effects

- **Brief 8.2 scenario** - A draws REVERSE while the queue holds B, C, D: the log prints REVERSE|A, ORDER|D|C|B; the next four turns are D, C, B, A (automated test).
- **Two players** - the queue holds one player; reversing it changes nothing and ORDER lists that one player.
- **PAY_EACH / COLLECT_EACH** after a REVERSE follow the new queue order, because brief 3.5 says "current turn-queue order" (tested).
- **Ranking ties** still use join order (brief 3.7), not queue order.
- **A double after REVERSE** - the same player rolls again in the same turn; the reversed order only matters when the turn ends.
- **A second REVERSE** restores the original direction; the card itself returns to the back of its deck like any other card.

3.3 T3 Expansion - growing the board at turn 20

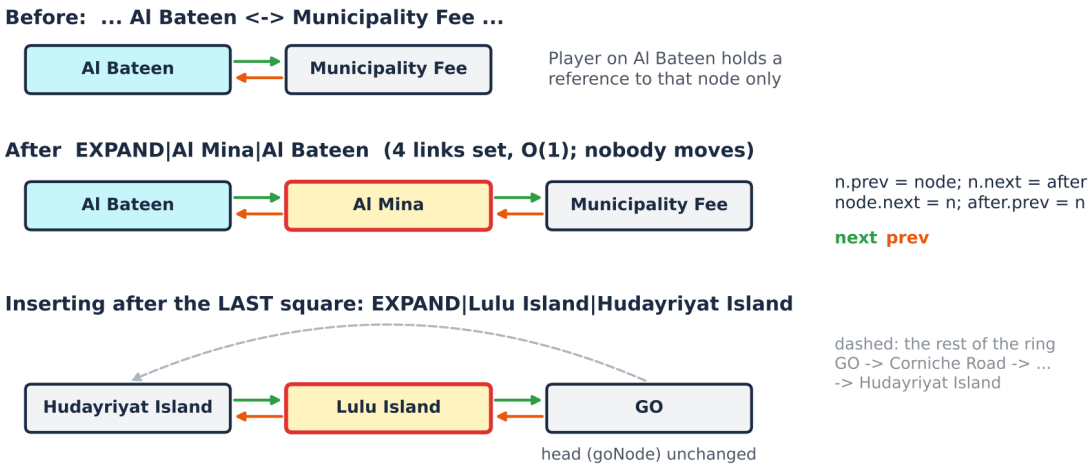
Rule (Appendix A, B.1). The board file may end with EXPAND lines: the name of an existing square, then a full square definition. They are not part of the starting board. At the end of turn 20 they are applied in file order, each new square inserted directly after the named square. Players are not moved. Log: EXPAND|<new square>|<after square>.

EXPAND line in code\data\board.txt	Why it is placed there
EXPAND|Al Bateen|PROPERTY|Al Mina|BLUE|80|6	BLUE becomes a 3-square group: an owner of Corniche Road and Al Bateen loses the doubled rent until they also buy Al Mina (ownsGroup walks the live ring)
EXPAND|Metro Green Line|TRANSIT|Abu Dhabi Airport Express|200	A fourth TRANSIT square: transitRent can now reach 25 x 2^3 = 200
EXPAND|Hudayriyat Island|PROPERTY|Lulu Island|RED|300|26	Inserted after the last square, so its next is GO and GO's prev becomes Lulu Island - the pointer-tracing case for the demo

How it works. `FileLoader` parses each EXPAND line with the same `parseSquare` used for normal squares and validates it at load time: the named square exists (or was added by an earlier EXPAND line), the new name is unique, the new square is not GO, JAIL or GO_TO_JAIL, EXPAND lines come last, and 24 + 3 stays within the 28-square limit. The lines wait on the Board in a `LinkedQueue<Expansion>` (file order = FIFO

order). At the end of turn 20, after the player has been re-enqueued and before the end-of-game check, the recursive `Game.applyExpansions` dequeues each line, finds the anchor node with the recursive findByName and calls insertAfter, which changes four links in O(1).

T3 Expansion - CircularDoublyLinkedList<Square>.insertAfter at the end of turn 20



Lulu.next = GO and GO.prev = Lulu, so step() still pays the salary when it reaches goNode; size() = 27, so searches and the board view adapt.

Code excerpt - CircularDoublyLinkedList.insertAfter and Game.applyExpansions

code\CircularDoublyLinkedList.java, code\Game.java

```
/** Inserts item directly after node and returns the new node. O(1). */
public Node<T> insertAfter(Node<T> node, T item) {
    if (node == null) {
        throw new IllegalArgumentException("insertAfter: node must not be null");
    }
    Node<T> n = new Node<>(item);
    Node<T> after = node.next;
    n.prev = node;
    n.next = after;
    node.next = n;
    after.prev = n;
    size++;
    return n;
}

/**
 * T3 Expansion, at the end of turn 20. Recursive over the pending queue
 * (file order): dequeue one EXPAND line, find the named square with the
 * recursive findByName, insert the new square directly after it, log
 * EXPAND|<new>|<after>. A later EXPAND line may name a square added by an
 * earlier one. Players are not moved.
 * Base case: no pending expansions. Terminates: one dequeue per call.
 */
public void applyExpansions() {
    LinkedQueue<Expansion> pending = board.getPendingExpansions();
    if (pending.isEmpty()) {
        return;
    }
    Expansion e = pending.dequeue();
    Node<Square> after = board.findByName(board.getHead(), e.getAfterName(), board.size());
    if (after == null) {
        throw new IllegalStateException("EXPAND: square '" + e.getAfterName() + "' is not on the board");
    }
    board.insertAfter(after, e.getSquare());
    logger.log("EXPAND", e.getSquare().getName(), e.getAfterName());
    applyExpansions();
}
```

Edge cases and knock-on effects

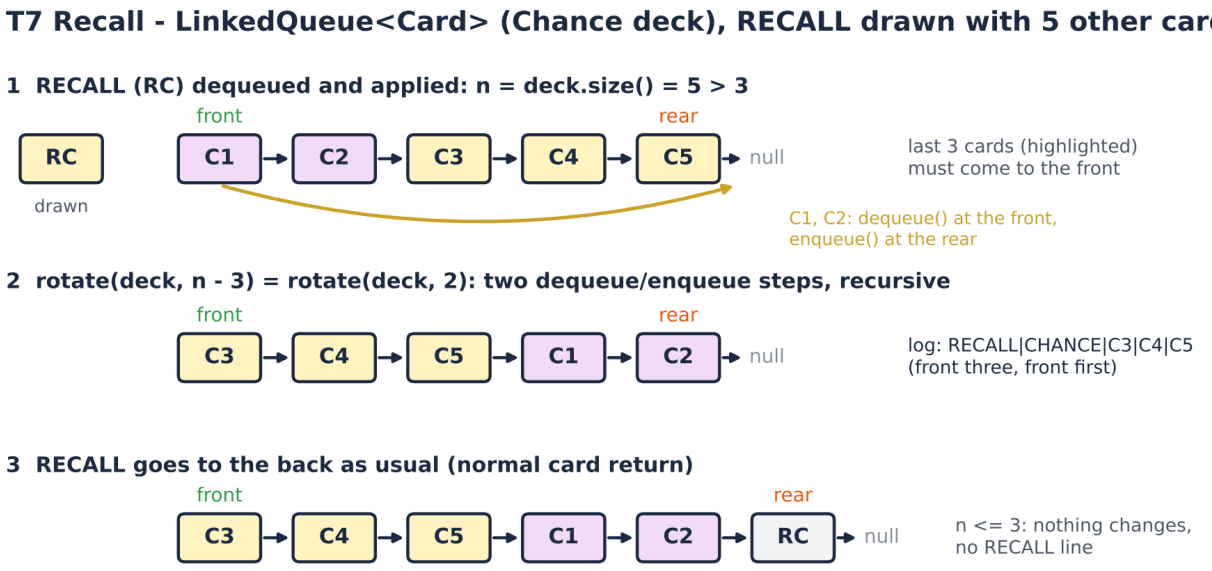
- **No player moves** - positions are node references; a player on Al Bateen still holds the same node, whose next is now Al Mina. Nothing is renumbered and no salary is paid (tested).

- **Inserting after the last square** - Lulu Island's next is GO and GO's prev is Lulu Island; the head (goNode) does not change, so step() still pays the salary when it reaches GO (tested: Hudayriyat +1 = Lulu Island, +2 = GO with salary).
- **Live size** - every search uses size() as its bound and the board view recomputes its layout, so the 27-square ring is drawn as an 8 x 8 ring (41 rows x 113 columns).
- **landOn termination** - the EXPAND positions were chosen so that no Chance card move (MOVE_BY -3 or +2) ends on a CHANCE or COMMUNITY square after the expansion; a test checks every card from every card square on both boards, so landOn depth stays at most 2.
- **Game ends before turn 20** - the expansion never happens. With --max-turns 20 the three EXPAND lines are printed just before END|MAX_TURNS.
- **MOVE_TO targets** are checked against the starting board when the cards load, so no card can point to a square that does not exist yet.

3.4 T7 Recall - bringing the last three cards to the front

Rule (Appendix A). After a RECALL card is drawn, the last 3 cards of the same deck move to the front, keeping their order. The RECALL card then goes to the back as usual. If the deck holds 3 or fewer cards apart from RECALL, nothing changes. It must be done by a recursive `static <T> void rotate(LinkedQueue<T> q, int k)` that moves k cards from the front to the back. Log: RECALL|<deck>|<id1>|<id2>|<id3>.

How it works. RECALL has already been dequeued when `Game.recall` runs, so `n = deck.size()` counts only the other cards (held JAIL_FREE cards are with players, not in the deck). Moving the first n - 3 cards to the back leaves the last 3 at the front in their original order. The three front ids are read with one copyInto rotation, which leaves the deck unchanged. applyCard then enqueues RECALL at the back.



Code excerpt - QueueTwists.rotate and Game.recall

code\QueueTwists.java, code\Game.java

```
/**
 * T7 Recall: moves k elements from the front of q to the back, one at a
 * time, keeping their order.
 * Base case: k <= 0 (or an empty queue) - nothing left to move.
 * Recursive case: dequeue the front, enqueue it at the back, then
 * rotate(q, k - 1).
 * Terminates because k falls by 1 each call.
 * Cost: O(k).
 */
public static <T> void rotate(LinkedQueue<T> q, int k) {
    if (k <= 0 || q.isEmpty()) {
        return;
    }
    q.enqueue(q.dequeue());
    rotate(q, k - 1);
}

private void recall(LinkedQueue<Card> deck, DeckType type) {
    int n = deck.size();
    if (n <= RECALL_CARDS) {
        return;
    }
    QueueTwists.rotate(deck, n - RECALL_CARDS);
    ArrayList<Card> cards = new ArrayList<>();
    deck.copyInto(cards);                    // one rotation, order unchanged
    logger.log("RECALL", type.name(), cards.get(0).getId(), cards.get(1).getId(), cards.get(2).getId());
}
```

Edge cases

- **n <= 3** - nothing changes and no RECALL line is printed: the log line needs three ids and the brief says nothing changes (design note D25, open question Q3).
- **n = 4** - rotate(deck, 1): the first card goes to the back; RECALL|deck|second|third|fourth (tested).
- **Held JAIL_FREE cards** are not in the deck and are not counted (tested: 3 cards in the deck plus one held card means no change).
- **RECALL drawn twice in a row** is impossible: RECALL always returns to the back of its deck.

3.5 How the twists change the standard flow

Point in the game	Standard flow (Section 3 of the brief)	With our twists
Loading the board	Every line becomes a square on the ring (addLast)	EXPAND lines are validated and held in a LinkedQueue<Expansion>; the starting board is still 24 squares
Drawing a card (landOn > CardSquare.land > drawCard > applyCard)	Dequeue, apply effect, enqueue at the back	REVERSE: reverse the turn queue, log REVERSE and ORDER. RECALL: rotate the deck, log RECALL. Both cards then return to the back as usual
End of a turn (playOneTurn)	Re-enqueue the player unless bankrupt; end-of-game check	Re-enqueue; if this was turn 20, insert the EXPAND squares and log EXPAND; then the end-of-game check
Turn order	Join order, rotating	Reversed by each REVERSE card; PAY_EACH and COLLECT_EACH follow the current order
Deck order	Shuffled once, then rotates one card per draw	A RECALL card moves the last 3 cards to the front
Board	Fixed 24 squares	27 squares from the end of turn 20; colour groups, transit counts, searches and the screen all use the live ring

3.6 Recursion in the twist methods

Method	Base case	Recursive case	Why it terminates	Cost
QueueTwists.reverse(q) (T1)	q.isEmpty(): return	x = dequeue(); reverse(q); enqueue(x)	Each call removes one element; empty after size() calls	O(n) operations, depth n
QueueTwists.rotate(q, k) (T7)	k <= 0 or q empty: return	enqueue(dequeue()); rotate(q, k - 1)	k falls by 1 each call	O(k)

Method	Base case	Recursive case	Why it terminates	Cost
Game.applyExpansions() (T3)	No pending expansion: return	Dequeue one line, insertAfter, log; applyExpansions()	One dequeue per call; the queue holds 3 lines	O(e x n) (search for each anchor)
Game.fillNames(order, i, f) (T1)	i >= size: return	f[i + 1] = name; fillNames(order, i + 1, f)	i rises to the list size	O(p)

3.7 Verification and demo

Test (tests\GameTests.java)	What it proves
reverse(1..5) = 5..1; empty and single unchanged; rotate(1..5, 2) = 3,4,5,1,2; rotate by 0 and by size	The two queue methods on their own, incl. that the rear pointer is intact afterwards
T1 8.2 scenario: A draws REVERSE, queue B, C, D	Exact log lines; queue D, C, B, A; next four turns D, C, B, A; card back at the bottom
T1 then PAY_EACH	Payments follow the reversed queue order
T7 with 5, 4 and 3 other cards; with a held JAIL_FREE card	RECALL line and resulting deck order; no change at n <= 3
T3 at turn 19 / 20; pointer checks; BLUE rent; salary across Lulu Island; --max-turns 20	Expansion only at the end of turn 20, in file order, four links per insertion, nobody moves, knock-on rules
T3 loader errors (6 cases)	Unknown anchor, second JAIL, duplicate name, EXPAND not last, more than 28 squares; chained EXPAND accepted
D9 card moves on both boards	No card move ends on a card square before or after the expansion (landOn depth <= 2)
1,000 seeded games (2-6 players, 200 turns)	No exceptions; invariants hold with all three twists active (500 games of 60 turns: 421 REVERSE and 404 RECALL events)

Demo command - one game that shows all three twists, with the same log every time (seed 73):

```
run.bat --players Sara,Omar,Ali,Huda --seed 73 --auto --text --max-turns 24 (game-log.txt excerpt)

TURN|3|Ali
ROLL|Ali|6|3
MOVE|Ali|GO|Chance
CARD|Ali|C12
RECALL|CHANCE|C3|C1|C2
...
TURN|8|Huda
ROLL|Huda|3|5
MOVE|Huda|Jail|Community Fund
CARD|Huda|K11
REVERSE|Huda
ORDER|Ali|Omar|Sara
TURN|9|Ali   (then 10 Omar, 11 Sara, 12 Huda)
...
TURN|20|Huda
ROLL|Huda|6|4
MOVE|Huda|GO|Al Reem Island
RENT|Huda|Sara|10|1454|1200
EXPAND|Al Mina|Al Bateen
EXPAND|Abu Dhabi Airport Express|Metro Green Line
EXPAND|Lulu Island|Hudayriyat Island
TURN|21|Ali
```

3.8 Design decisions and open questions

Ref.	Decision or question
D22	T1 reverses only the waiting players; the current player is enqueued at the back by playOneTurn as usual. ORDER is read after the reversal with one copyInto rotation.
D23	After T1, PAY_EACH / COLLECT_EACH follow the new queue order (brief 3.5); ranking ties still use join order (brief 3.7).
D24	T7: n = deck.size() after RECALL is dequeued; rotate(deck, n - 3); RECALL returns to the back by the normal card return (D14).
D25	T7 with n <= 3: no change and no RECALL line. Held JAIL_FREE cards are not counted.
D26	T3: EXPAND lines validated at load time, held in a LinkedQueue<Expansion>, applied at the end of turn 20 after the re-enqueue and before the end-of-game check.
D27	T3 knock-ons: no player moves; colour groups and transit counts use the live ring; Lulu Island after the last square links to GO; size() is live.

Ref.	Decision or question
Q1	Twists: CS chose T1, T3, T7 (Conversation 03). To be checked against the group's Blackboard list - an earlier teammate log mentioned T1, T5, T8.
Q3	T7: should a RECALL line be printed when nothing changes? (implemented: no)
Q4	T3: is the expansion applied before the end-of-game check of turn 20? (implemented: yes)

4. UML class diagram (v4, as built)

Rotated to fit the page; full-size versions: design\UML_Class_Diagram.svg and .png. The Twists package (QueueTwists, Expansion) and the new members of Board, Game and CardEffect were added for T1, T3 and T7.

